

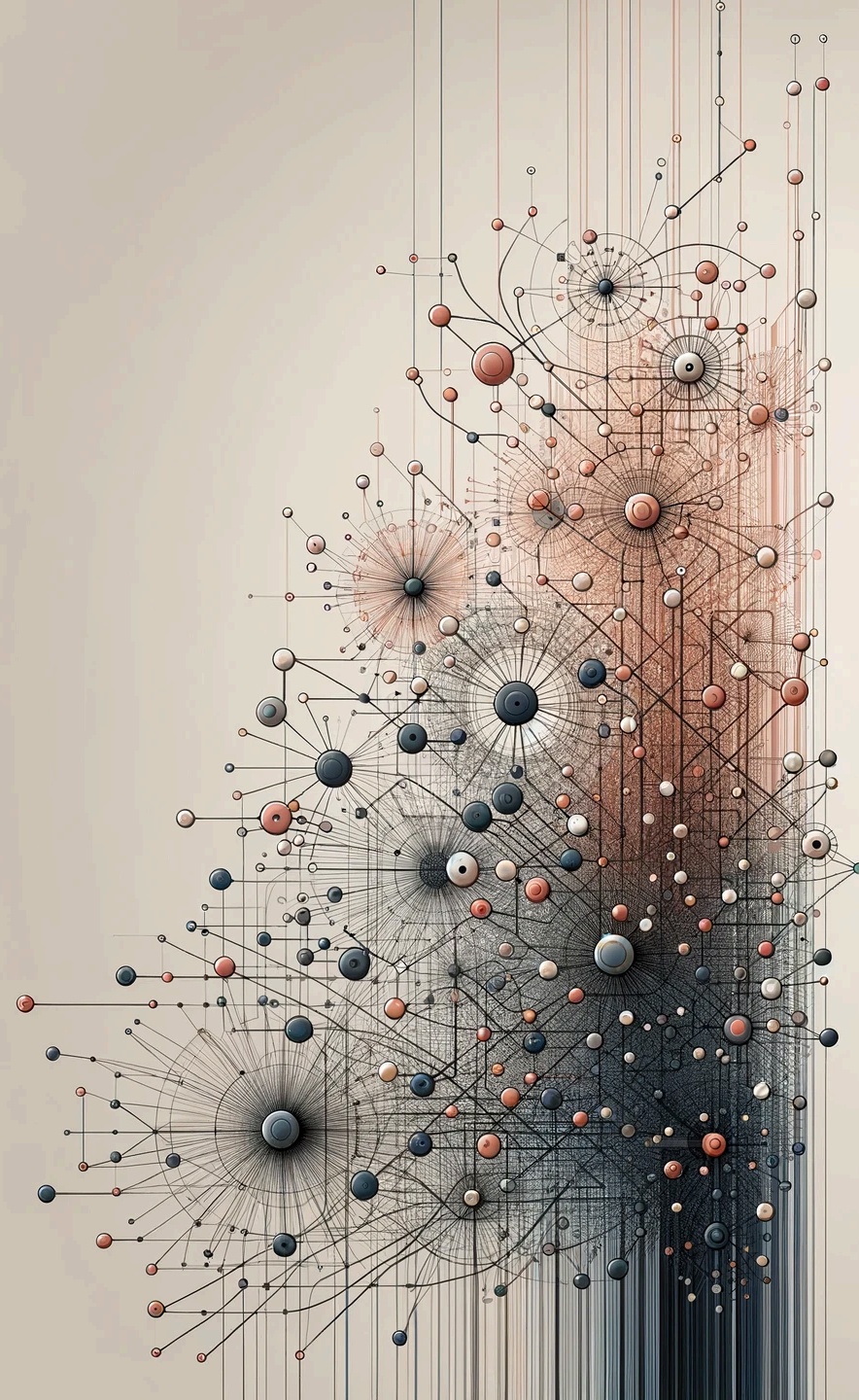


UNIVERSITÀ  
DI TORINO

# Analisi e Visualizzazione delle Reti Complesse

## NS21 - Representation Learning on Graphs

Prof. Rossano Schifanella



## Outline

1. **Foundations** — from hand-crafted features to learned representations
2. **The encoder–decoder framework** — a unified view of graph embeddings
3. **Defining node similarity** — proximities, walks, and what they capture
4. **Shallow embeddings I — DeepWalk** — random walks meet Skip-gram
5. **Shallow embeddings II — node2vec** — biased walks and exploration control
6. **Alternative families** — matrix factorization, spectral, LINE, autoencoders
7. **Graph-level embeddings** — pooling, supernodes, hierarchical methods
8. **Evaluation in practice** — link prediction, node classification, visualization
9. **Limitations and the bridge to Graph Neural Networks**



# From hand-crafted features to learned representations

## Traditional feature engineering

For decades, applying machine learning to networks meant **designing features by hand**:

- Pick a list of structural descriptors: degree, clustering coefficient, betweenness, PageRank, k-core number, motif counts, ...
- Compute them for every node (or pair of nodes, for link tasks)
- Feed the resulting vectors into classical models — logistic regression, SVMs, random forests, gradient boosting

**Worked example — predicting influential users in a social network:**

1. Compute degree, PageRank, community membership, eigenvector centrality for every account
2. Stack them into a feature matrix  $X \in \mathbb{R}^{|V| \times k}$
3. Train an SVM on a labelled subset of "influential / not influential" users
4. Predict the rest

This works — but every step in the pipeline depends on **human intuition** about which descriptors matter.

## Why hand-crafted features fall short

- **Labour-intensive:** a new task means a new round of feature engineering
- **Domain expertise required:** knowing what to compute presupposes knowing the network well
- **Poor transfer:** features designed to predict influence rarely also help predict community membership or future links
- **Ceiling on expressiveness:** simple statistics cannot capture intricate, multi-hop dependencies that often drive behaviour
- **Brittle to network type:** descriptors that work for social graphs may be useless for molecules or roads

**Concrete example.** In protein–protein interaction networks, two proteins can have very different degrees yet share the same biological function because they sit in equivalent **structural positions**. No single hand-crafted statistic captures this — but a learned embedding can.

## A new paradigm — *learn* the features

Representation learning flips the pipeline:

- We **do not pre-specify** which descriptors matter
- We let an algorithm **discover** a vector representation  $\mathbf{z}_v \in \mathbb{R}^d$  for each node directly from the graph
- The same representation can then be used for **many downstream tasks**: classification, link prediction, clustering, recommendation, anomaly detection

Two consequences worth emphasising:

1. **Features emerge from the data**, not from human guesswork
2. **A single embedding, many uses** — the same vectors can power a recommender system today and a fraud detector tomorrow

The rest of this lecture builds up the machinery that makes this possible.

## What is an embedding?

An **embedding** is a mapping from a discrete object (a word, an image, a node, a graph) to a vector of real numbers, with the property that **objects judged "similar" by some criterion end up close in vector space**.

Once we have such a mapping, the embedding vectors become a drop-in input to any machine learning algorithm — k-NN, logistic regression, neural networks — none of which know anything about the original graph.

Three families of graph embeddings exist, distinguished by *what* is embedded:

- **Node embeddings** — one vector per node (*the focus of this lecture*)  
Applications: node classification, link prediction, recommendation, clustering
- **Edge / relation embeddings** — one vector per edge or per relation type  
Applications: knowledge graphs (Freebase, WordNet), multi-relational networks
- **Graph embeddings** — one vector for an entire graph  
Applications: molecule property prediction, graph classification, graph similarity search



# Mathematical foundations



## Embedding as dimensionality reduction

A graph on  $|V|$  nodes lives in a space of dimension  $|V|$ :

- The adjacency matrix  $A \in \{0, 1\}^{|V| \times |V|}$  has  $O(|V|^2)$  entries
- Each row of  $A$  is the "raw" representation of a node — a binary vector saying *who you connect to*
- For a million-node graph, this is a million-dimensional, mostly-zero vector per node

We want a compact representation: a matrix  $Z \in \mathbb{R}^{|V| \times d}$  with  $d \ll |V|$  (typically  $d = 64, 128, 256$ ) such that the geometry of the rows of  $Z$  reflects the **graph-specific structure** we care about.

This is dimensionality reduction in spirit — but unlike PCA on tabular data, we cannot just minimise reconstruction error of  $A$ , because  $A$  encodes a relational object whose meaningful structure is multi-hop, hierarchical, and often global.

## What does "similar nodes" mean? — Proximity orders

Different embedding methods make different choices about which kind of similarity to preserve. The standard taxonomy:

- **First-order proximity** — direct edges  
*Connected nodes should have similar vectors.*  
Example: friends in a social network.  
Captures: the immediate neighbourhood — who you know directly.
- **Second-order proximity** — shared neighbourhoods  
*Nodes with many common neighbours should be similar, even if not directly connected.*  
Example: two professors who collaborate with the same researchers but never with each other.  
Captures: roles, contexts, "friends-of-friends" patterns.
- **Higher-order / global structure** — multi-hop paths  
*Preserve overall network distances, communities, hierarchical organisation.*  
Captures: position within the network as a whole — bridges, peripheries, hubs.

The choice of proximity is the **most consequential design decision** in any embedding method — it determines which downstream tasks the embeddings will be good at.

## The encoder–decoder framework

Almost every node-embedding method — shallow or deep — fits the same template, made up of four components:

1. **Encoder**  $ENC : V \rightarrow \mathbb{R}^d$ ,  $ENC(v) = \mathbf{z}_v$

Maps each node to a  $d$ -dimensional vector.

2. **Graph similarity function**  $s_G(u, v)$

The *target* — what "similar" means in the original graph.

Examples:  $A_{uv}$  (adjacency), shared-neighbour count, shortest-path distance, Katz index, random-walk co-occurrence probability.

3. **Decoder**  $DEC(\mathbf{z}_u, \mathbf{z}_v) \approx s_G(u, v)$

Reconstructs the graph similarity from the two embeddings.

Most common choice: the dot product  $\mathbf{z}_u^\top \mathbf{z}_v$ .

4. **Optimisation objective**

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \ell(DEC(\mathbf{z}_u, \mathbf{z}_v), s_G(u, v))$$

where  $\ell$  is a per-pair loss (squared error, cross-entropy, etc.).

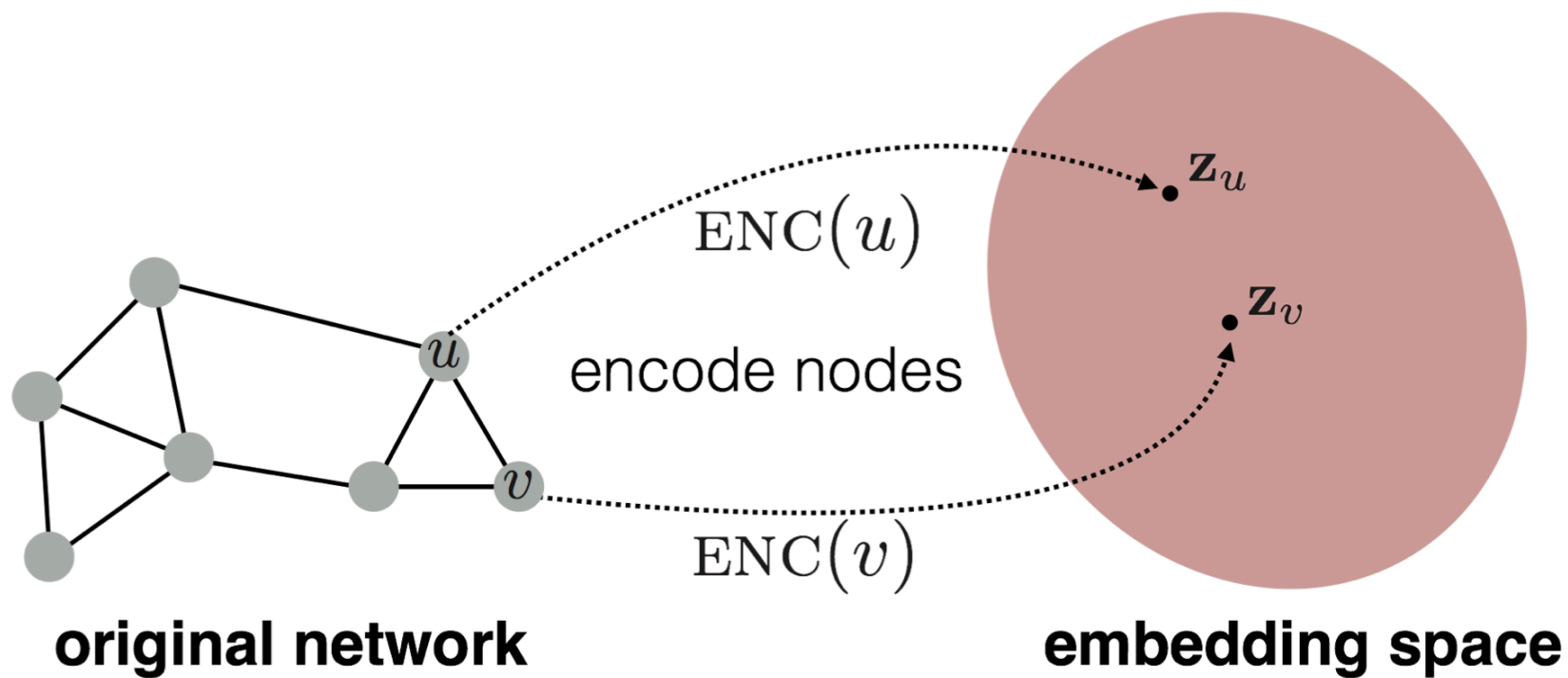
## Reading the framework

The framework is a recipe with three knobs:

- **Choose a similarity**  $s_G \rightarrow$  fixes *what the embeddings will preserve*
- **Choose an encoder**  $ENC \rightarrow$  fixes *how nodes are mapped to vectors* (lookup table? neural network?)
- **Choose a decoder + loss**  $\rightarrow$  fixes *how mismatch is measured and corrected*

Almost every published method — DeepWalk, node2vec, LINE, HOPE, GraphSAGE, GCN, GAT, GIN — is a particular instantiation of these three choices.

The slides that follow walk through the most influential instantiations, starting from the simplest possible encoder.



## Two families of encoders — shallow vs. deep

### Shallow encoders (*the focus of this lecture*)

- Direct mapping: each node ID indexes a row of an embedding matrix  $Z$
- Each node has its own independent parameter vector — no sharing
- Total parameters:  $O(|V| \cdot d)$
- Examples: DeepWalk, node2vec, LINE, HOPE

### Deep encoders (*covered in the next lecture*)

- A neural network computes the embedding from node features and structural information
- Parameters are **shared** across nodes, so the model size does not grow with  $|V|$
- Total parameters:  $O(d^2)$  (independent of graph size)
- Examples: Graph Convolutional Networks (GCN), GraphSAGE, Graph Attention Networks (GAT), GIN

## Shallow vs. deep encoders — comparative table

| Aspect                 | Shallow Encoders                       | Deep Encoders                        |
|------------------------|----------------------------------------|--------------------------------------|
| Training               | Faster, simpler                        | More complex, requires tuning        |
| Parameters             | $O( V  \times d)$                      | $O(d^2)$ (independent of graph size) |
| Node features          | Ignored                                | Fully leveraged                      |
| Inductive?             | No (transductive only)                 | Yes                                  |
| Scalability bottleneck | Memory ( $Z$ grows with $ V $ )        | Computation (message passing)        |
| Best for               | Static graphs, simple downstream tasks | Dynamic graphs, feature-rich domains |

## Inductive vs. transductive — a crucial distinction

**Transductive embeddings** are learned for the specific nodes seen during training. The embedding of node  $v$  is a parameter — it does not exist for any node not in  $V$ . Adding a new node requires **retraining**.

*Example methods:* DeepWalk, node2vec, spectral embeddings, LINE.

**Inductive embeddings** learn a *function* — typically a neural network that takes a node's features and neighbourhood and produces an embedding. The same function can be applied to **unseen nodes** without retraining.

*Example methods:* GraphSAGE, GAT, GIN.

**Why this matters in practice.** Real systems evolve constantly — new users sign up, new molecules are synthesised, new web pages appear. A transductive method must be retrained from scratch each time; an inductive method just runs the encoder on the new input. The cost difference is enormous.





## Defining node similarity

## Shallow encoding — the embedding lookup

The simplest possible encoder is a **lookup table**:

- Stack all embeddings into a matrix  $Z \in \mathbb{R}^{|V| \times d}$
- $ENC(v) = Z[v]$  — return row  $v$

**What we are optimising.** Unlike a neural network, the parameters *are* the embedding values themselves. There is no internal layer that produces  $\mathbf{z}_v$  —  $\mathbf{z}_v$  is a parameter, learned directly by gradient descent.

### Consequences:

- **No parameter sharing** — updating  $\mathbf{z}_u$  does not affect  $\mathbf{z}_v$
- **Cannot use node features** — the encoder sees only the node ID
- **Transductive** — there is no rule that produces an embedding for a node not seen at training time
- **Simple and effective** for moderate-sized static graphs

The interesting design question shifts from *the encoder* to *the similarity function*.

## From graph theory to a learning target

The encoder–decoder objective requires a concrete  $s_G(u, v)$ . The choice determines what kind of "similarity" the embeddings will encode.

| Similarity measure                      | Mathematical form                        | What it captures             | Picture                       |
|-----------------------------------------|------------------------------------------|------------------------------|-------------------------------|
| <b>First-order</b> (adjacency)          | $A_{uv}$                                 | direct connection            | $u \text{---} v$              |
| <b>Second-order</b> (common neighbours) | $\sum_w A_{uw} A_{wv} = (A^2)_{uv}$      | shared neighbourhood         | $u \text{---} w \text{---} v$ |
| <b>Higher-order</b> ( $k$ -step paths)  | $(A^k)_{uv}$                             | multi-hop connectivity       | paths of length $k$           |
| <b>Katz index</b>                       | $\sum_{k=1}^{\infty} \beta^k (A^k)_{uv}$ | all paths, decayed by length | weighted infinite sum         |
| <b>Personalised PageRank</b>            | restart probability from $u$ to $v$      | random-walk influence        | stationary distribution       |

## The scalability problem

Recall the loss

$$\mathcal{L} = \sum_{u,v \in V} \| DEC(\mathbf{z}_u, \mathbf{z}_v) - s_G(u, v) \|^2.$$

Two costs blow up on real graphs:

1. **The sum has  $|V|^2$  terms** — for a million-node graph, that is  $10^{12}$  pairs per gradient step.
2. **Computing  $s_G(u, v)$  itself can be expensive** — Katz needs an infinite series, personalised PageRank needs a power iteration per source node.

For graphs with millions of nodes, computing the loss exactly is **infeasible**. We need an approximation.

## Sampling — the way out

Rather than enumerate all pairs, we **sample**:

1. **Positive pairs** — node pairs that are *likely* similar under  $s_G$
2. **Negative pairs** — random pairs that are *likely* dissimilar
3. **Train the embeddings to score positives high and negatives low**

Two key ideas make this work:

- We need a cheap way to **generate positive pairs** that respect the chosen similarity  
→ **random walks**
- We need a cheap loss that does not require summing over all  $|V|$  nodes per update  
→ **negative sampling**

The next sections develop both.

## Random walks — definition and intuition

A **random walk** on a graph  $G$  is a stochastic process:

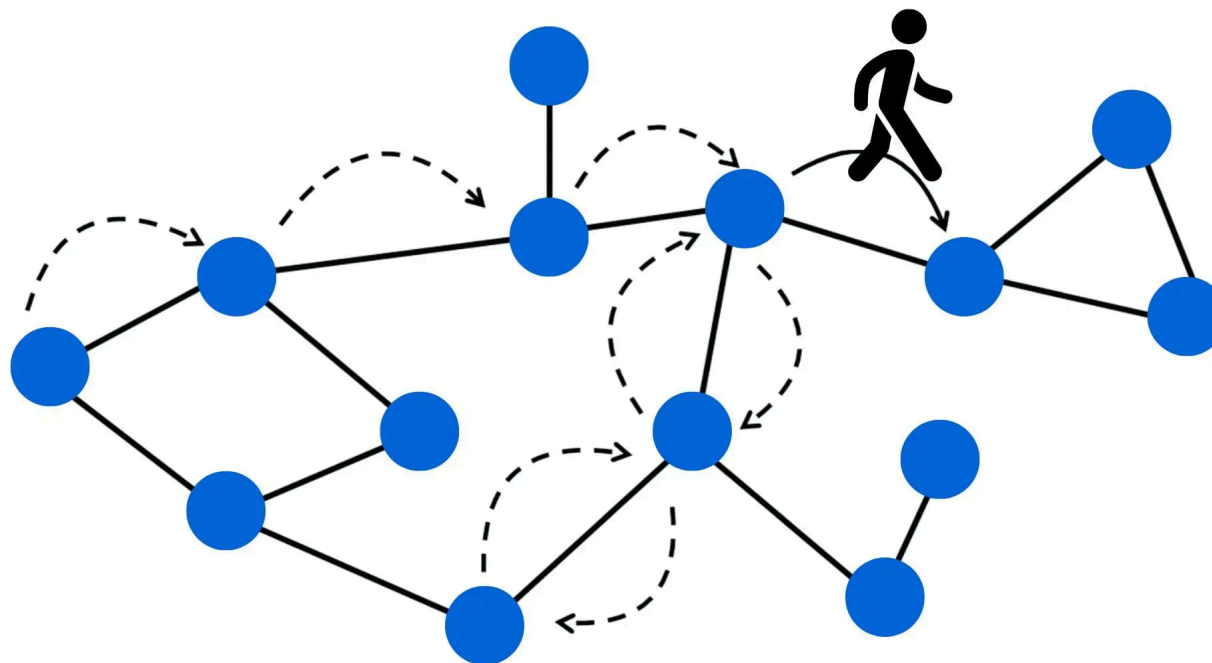
- Start at a node  $u_0$
- At step  $t$ , jump to a neighbour of the current node  $u_t$ , chosen according to some transition rule
- Record the visited sequence  $u_0, u_1, u_2, \dots, u_L$

The walk's length  $L$  controls how far it explores; the transition rule controls *what* it explores (uniform neighbour, biased, weighted, ...).

### Why walks define a good similarity.

1. **Flexible context** — nodes that co-occur in walks are operationally "related"; the definition adapts to the graph
2. **Local + global balance** — short walks stay in communities, long walks reach distant structural roles
3. **Cheap to compute** — generating one walk costs  $O(L)$ , no matrix inverses, no eigendecompositions

4. **Implicit higher-order information** — co-occurrence at distance  $k$  in a walk encodes a  $k$ -step relationship without computing  $A^k$



## From walks to a learning objective

For each node  $u$ , define its **network neighbourhood**  $N_R(u)$  as the multiset of nodes that appear within a window of  $u$  across many random walks rooted at (or passing through)  $u$ .

The optimisation problem becomes:

$$\max_f \sum_{u \in V} \log P(N_R(u) \mid f(u)) ,$$

where  $f(u) = \mathbf{z}_u$  is the embedding function.

In words: **choose embeddings that make each node's observed walk-neighbourhood maximally probable.**

The next slide makes the analogy to language explicit — it is the conceptual leap that unlocked the entire field.



## The word–node analogy and the Skip-gram objective

Word embeddings (Word2Vec, Mikolov et al. 2013) learn vectors for words by predicting which words appear together in sentences. The same idea, transplanted to graphs, becomes node embeddings.

### Two parallel pipelines:

| Language                                        | Networks                                      |
|-------------------------------------------------|-----------------------------------------------|
| Sentence — sequence of words                    | Random walk — sequence of nodes               |
| Context window around a target word             | Window around a target node in the walk       |
| "Bank" near "money" / "loan" / "river"          | A person near friends / colleagues / family   |
| Distributional hypothesis: meaning from context | Structural hypothesis: role from walk-context |

*"You shall know a word by the company it keeps" (Firth, 1957)*

*"You shall know a node by the walks it participates in"*

## Word2Vec primer — Skip-gram in one slide

The **Skip-gram** model (Mikolov et al., 2013) learns vectors by solving a single task:

*Given a target node, predict the nodes that appear near it in a walk.*

The objective, written in words:

$$\text{maximise} \quad \sum_{(\text{target}, \text{context}) \text{ pairs}} \log P(\text{context} \mid \text{target}),$$

where the probability is computed from the dot product  $\mathbf{z}_u^\top \mathbf{z}_v$  — a higher dot product means a higher predicted probability of co-occurrence.

**The bottleneck.** Computing this probability *exact/y* requires comparing each target against **every node in the graph** at every step. For graphs with millions of nodes, that is infeasible. The next slide shows the standard trick that fixes this.

## Negative sampling — making it tractable

**The trick.** Instead of asking "*out of all  $|V|$  nodes, which one comes next?*", we ask many smaller yes/no questions:

- For each **true** (target, context) pair from the walks — say "*yes, these two are similar*" and push their embeddings **closer**.
- For a handful of **random** node pairs (say  $k = 5$  to  $20$ ) — say "*no, these two are not similar*" and push their embeddings **apart**.

That is it. We never compute the full softmax denominator; we just contrast positives against a few random negatives.

**Cost per update:** proportional to  $k$  instead of to  $|V|$ . With  $k = 10$ , training is roughly **100,000 times cheaper** on a million-node graph.

This single trick is what makes Skip-gram (and therefore DeepWalk and node2vec) **scale to millions of nodes**.

## Other families of node-embedding methods

Random walks are popular but not the only route. Three other major families exist; each makes a different choice in the encoder–decoder framework.

### 1. Matrix factorisation methods (*GF, HOPE, GraRep*)

- Build a node-similarity matrix  $S$  explicitly (adjacency,  $A^k$ , Katz, ...)
- Find  $Z$  such that  $ZZ^\top \approx S$  — solved with classical linear algebra (e.g. SVD)
- Surprising result: DeepWalk turns out to be doing this *implicitly* on a particular similarity matrix (Qiu et al. 2018) — the random-walk and matrix-factorisation views are two sides of the same coin

### 2. Spectral methods (*Laplacian Eigenmaps*)

- Compute the eigendecomposition of the graph Laplacian  $L = D - A$
- Use the eigenvectors corresponding to the smallest non-zero eigenvalues as embeddings
- Preserve global graph properties; closely related to spectral clustering

### 3. Direct first/second-order methods — LINE (*Large-scale Information Network Embedding, Tang et al. 2015*)

Unlike DeepWalk, LINE skips random walks entirely. It directly trains two separate embeddings per node:

- one that pushes **directly connected nodes** close (first-order proximity)
- one that pushes **nodes with shared neighbours** close (second-order proximity)

The two are concatenated into the final representation. LINE was one of the first methods to operationalise the proximity orders directly, and it scales to graphs with billions of edges.

#### 4. Deep autoencoder approaches (*SDNE*, *DNGR*)

- Compress and reconstruct adjacency rows (or PPMI matrices) with a deep autoencoder
- Hidden layer = embedding
- Captures non-linear structure but shares the transductive limitation of shallow methods

These four families — random walks, matrix factorisation, spectral, autoencoders — together cover almost the entire pre-GNN landscape. Each is a different combination of *what to preserve* and *how to optimise*.



## Random-walk methods in practice

## The lineage of random-walk embeddings

The theory of random walks for embeddings translates into a small family of practical algorithms that differ mainly in **how they generate walks** and **how they define neighbourhoods**:

- **DeepWalk** (Perozzi et al., 2014) — first to combine uniform random walks with Skip-gram
- **node2vec** (Grover & Leskovec, 2016) — adds two parameters  $p, q$  to bias the walk between BFS-like and DFS-like behaviour
- **LINE** (Tang et al., 2015) — direct first/second-order objectives, no walks
- **HARP** — coarsens the graph hierarchically before running DeepWalk/node2vec
- **VERSE** — uses arbitrary similarity matrices (PPR, SimRank, ...) instead of walks

We focus on DeepWalk and node2vec — the two methods that defined the field.



## DeepWalk

- **Origin** — Perozzi, Al-Rfou, Skiena (KDD 2014); the first major graph-embedding method
- **Core idea** — nodes appearing in similar contexts in random walks should have similar embeddings
- **Inspiration** — directly transplants Word2Vec's Skip-gram from text to graphs
- **Analogy** — a random walk is a "sentence", a node is a "word"
- **Reference** — [DeepWalk: Online Learning of Social Representations \(KDD 2014\)](#)

The algorithm has only three pieces — walk generation, context windowing, and Skip-gram training — but the combination was a breakthrough at the time, demonstrating that an off-the-shelf NLP model could match or beat purpose-built graph methods.

## DeepWalk — step by step

### Step 1 — Generate random walks.

For each node  $v$  in the graph:

- Start a walk from  $v$
- For  $L$  steps, move to a uniformly random neighbour: transition probability  $\frac{1}{\deg(\text{current})}$
- Repeat  $r$  times per node

The original paper used  $r = 80$  walks per node (Perozzi et al., §6.1); modern reimplementations and node2vec settle on  $r = 10$ ,  $L = 80$ .

Result: a *corpus* of  $r \cdot |V|$  node sequences — the analogue of a text corpus.

### Step 2 — Build context pairs.

For each walk and each position  $i$ :

- Slide a window of half-width  $w$  (typically  $w = 5$  or  $10$ ) over the walk
- Treat  $(v_i, v_j)$  for  $j \in [i - w, i + w] \setminus \{i\}$  as a (target, context) training pair

Each walk of length  $L$  yields roughly  $2wL$  pairs.

## DeepWalk — the learning step

### Step 3 — Apply Skip-gram.

- Initialise the embedding matrix  $Z$  randomly
- For each (target, context) pair from the corpus of walks, nudge their embeddings *closer*; for a few random non-context nodes, nudge them *apart* (negative sampling, previous slide)
- Repeat over many passes until embeddings stabilise

**Historical note.** The original DeepWalk paper (2014) used a different trick called **hierarchical softmax** to dodge the same bottleneck. node2vec (2016) switched to negative sampling, which is now the standard for both methods.

### Step 4 — Output.

Each row of the trained matrix  $Z$  is a  $d$ -dimensional embedding, ready to be plugged into any downstream classifier or similarity computation.

## DeepWalk — hyperparameters and what they control

| Hyperparameter            | Original paper | Common modern default | What it controls                          |
|---------------------------|----------------|-----------------------|-------------------------------------------|
| $d$ — embedding dimension | 128            | 64–256                | Capacity vs. overfitting; downstream cost |
| $L$ — walk length         | 40             | 40–80                 | How far context can reach                 |
| $r$ — walks per node      | 80             | 10–20                 | Statistical coverage of the corpus        |
| $w$ — context window      | 10             | 5–10                  | What counts as "near" in a walk           |
| $k$ — negative samples    | n/a (HSM)      | 5–20                  | Cost of each gradient step (if SGNS used) |

## DeepWalk — strengths

- **Conceptual simplicity** — one short paper, one short algorithm
- **Scales** to graphs with millions of nodes thanks to walks + negative sampling
- **No feature engineering** — only the graph structure is needed
- **Versatile** — performs reasonably on social, citation, biological, and web graphs
- **Drop-in vectors** — outputs feed directly into any sklearn-style classifier

## DeepWalk — limitations

- **Uniform exploration** — every neighbour is equally likely; no way to favour BFS- or DFS-like exploration
- **No edge-type awareness** — every edge is the same; cannot exploit edge labels or weights without modification
- **No node features** — rich metadata (text, images, attributes) is ignored
- **Transductive** — the embedding for a new node does not exist; retraining is required
- **Static** — designed for a fixed snapshot; evolving graphs need workarounds
- **Few knobs** — limited ability to adapt the algorithm to different graph types or tasks

These limitations directly motivated the next algorithm.

## node2vec

### What changed

- **Origin** — Grover & Leskovec (KDD 2016); explicitly designed to fix DeepWalk's "one-size-fits-all" walk strategy
- **Core innovation** — **biased** random walks with two interpretable parameters  $p$  and  $q$  that smoothly interpolate between BFS-like and DFS-like exploration
- **Reference** — [node2vec: Scalable Feature Learning for Networks \(KDD 2016\)](#)

The training pipeline (Skip-gram + negative sampling) is **identical** to DeepWalk. The novelty is entirely in **how walks are generated**.

## Two structural properties node2vec can capture

The mapping between walk strategy and what gets preserved is **counter-intuitive** — read carefully (Grover & Leskovec, 2016, §3.1).

### 1. Structural equivalence (functional roles).

Nodes playing similar *roles* in different parts of the graph should be embedded close together — even if they are far apart in the network.

*Example.* Two airports, one in Europe and one in Asia, that both serve as international hubs have similar local connection patterns despite no direct link between them.

*Captured by* **BFS-like, local exploration** — by restricting walks to the immediate neighbourhood, the algorithm samples the local "fingerprint" of each node, and nodes with similar fingerprints end up close in embedding space.



## 2. Homophily (community structure).

Nodes in the same community should be embedded close together — cliques of friends, dense subgraphs, topical clusters.

*Captured by **DFS-like, broader exploration*** — long, outward-bound walks reflect a macro-view of the neighbourhood, so nodes that belong to the same dense region tend to co-occur in the same walks.

The single parameter pair  $(p, q)$  lets the practitioner choose where on this spectrum to operate.

## node2vec — the algorithm

### Walk control parameters.

- **Return parameter**  $p$  — controls the chance of stepping back to the previous node
  - $p < 1$ : walks revisit (stay in tight neighbourhoods)
  - $p > 1$ : walks avoid backtracking (explore outward)
- **In-out parameter**  $q$  — controls "outward" vs. "inward" exploration
  - $q < 1$ : walks venture outward (DFS-like — captures **homophily / communities**)
  - $q > 1$ : walks stay near the source (BFS-like — captures **structural equivalence / roles**)

### Pipeline.

1. Pick  $p$  and  $q$  for your task
2. For each node, run  $r$  biased random walks of length  $L$
3. Apply Skip-gram with negative sampling to the resulting walk corpus — exactly as DeepWalk does

## The biased transition rule

A node2vec walk has **memory of the previous step**. Suppose the walk just moved from  $t$  to  $v$  and now needs to pick the next node  $x$  from  $v$ 's neighbours. There are exactly three kinds of moves:

| Move type | Description                               | Probability is multiplied by |
|-----------|-------------------------------------------|------------------------------|
| Return    | step back to $t$                          | $1/p$                        |
| Triangle  | jump to a node also adjacent to $t$       | 1                            |
| Outward   | jump to a node <i>not</i> adjacent to $t$ | $1/q$                        |

In other words: one knob ( $p$ ) controls *how willing the walk is to backtrack*; the other ( $q$ ) controls *how willing it is to wander away* from where it came from. The case  $p = q = 1$  recovers DeepWalk's plain uniform walk.

### Special cases.

- $p = q = 1 \rightarrow$  uniform walk  $\rightarrow$  **DeepWalk**

- $q \gg 1$  (BFS-like, walk stays local)  $\rightarrow$  **structural equivalence / roles**

- $p \gg 1$  (DFS-like, walk stays local)  $\rightarrow$  **structural equivalence / roles**

## Worked example — what the bias does

Suppose we set  $p = 0.25$ ,  $q = 4$  (small  $p$ , large  $q \rightarrow$  strong **return**, weak **outward**).

From a node  $v$  that has just been reached from  $t$ , the walk has three options. The bias makes them very unequal:

- **Going back to  $t$**  is multiplied by  $1/p = 4 \rightarrow$  very likely
- **Stepping to a triangle neighbour** stays at weight 1  $\rightarrow$  moderately likely
- **Wandering outward** is multiplied by  $1/q = 0.25 \rightarrow$  unlikely

Result: the walk **stays glued to the immediate neighbourhood** — a BFS-like regime that samples each node's local fingerprint thoroughly. Per the original paper, this is what surfaces **structural equivalence** (e.g. two hub airports in different countries with similar local patterns).

Flipping the values to  $p = 4$ ,  $q = 0.25$  inverts the picture: backtracking is discouraged, outward exploration is encouraged, and the walk roams the network — revealing **communities (homophily)**.

## Choosing the strategy by task

Settings used in the original paper's Les Misérables case study (Grover & Leskovec, §4.1):

| Task                                       | Suggested<br>$p, q$ | Walk behaviour                         | Preserves                    |
|--------------------------------------------|---------------------|----------------------------------------|------------------------------|
| Community detection<br>(homophily)         | $p = 1, q = 0.5$    | DFS-like, broad<br>outward exploration | Macro-view →<br>communities  |
| Role discovery (structural<br>equivalence) | $p = 1, q = 2$      | BFS-like, walks stay<br>local          | Local fingerprint →<br>roles |
| Link prediction                            | task-<br>dependent  | Use cross-validated<br>grid search     | Mix of both                  |

**The advantage over DeepWalk** is precisely this tunability: the same algorithm can be configured to surface community structure or structural equivalence, addressing DeepWalk's lack of control with two interpretable knobs.



# From nodes to whole graphs

## Use case — molecule classification

- Each molecule is a small graph — atoms are nodes, bonds are edges
- We want to predict properties of the molecule as a whole: toxicity, solubility, reactivity, drug-likeness
- The challenge: **molecules vary in size**, so we need a *fixed-dimensional* representation that does not depend on  $|V|$

This is the prototypical **graph-level** prediction problem, and the same techniques apply to social subgraph classification, scene-graph recognition, code-property graphs, and many other domains.

## Method 1 — aggregate node embeddings

The simplest approach: compute node embeddings, then **pool**:

- **Sum pooling:**  $\mathbf{z}_G = \sum_{v \in V_G} \mathbf{z}_v$
- **Mean pooling:**  $\mathbf{z}_G = \frac{1}{|V_G|} \sum_{v \in V_G} \mathbf{z}_v$
- **Max pooling:**  $\mathbf{z}_G = \max_{v \in V_G} \mathbf{z}_v$  (element-wise)

Each has trade-offs: sum is sensitive to size, mean is size-invariant, max captures the most "extreme" features. In practice, the choice matters.

## Method 2 — virtual supernode

1. Add a dummy node  $s$  connected to **every** node in the graph
2. Run any node-embedding algorithm
3. Use  $\mathbf{z}_s$  as the graph representation

The supernode "sees" the whole graph during training, so its embedding aggregates global information automatically.



## Advanced graph-level methods (*out of scope, for reference*)

Beyond simple aggregation, several families capture richer graph-level structure:

### Hierarchical pooling.

Progressively coarsen the graph:  $Z^{(l+1)} = \text{POOL}(Z^{(l)}, A^{(l)})$ . Produces a **multi-resolution** representation. Examples: DiffPool, SAGPool.

### Graph kernels.

Compare substructures (graphlets, random walks, subtrees) directly. The Weisfeiler–Lehman kernel iteratively relabels nodes from neighbour multisets and counts label co-occurrences. Implicitly defines an embedding via the kernel matrix.

## **Self-supervised methods.**

Train a model to predict graph-level properties (size, diameter, motif counts) or to reconstruct masked subgraphs. Learns rich representations without task labels.

These methods are typically built on top of GNNs, which makes them part of the next lecture rather than this one.



# Evaluating embeddings

## How do we know an embedding is "good"?

Embeddings are intermediate objects — their quality is judged by **downstream task performance**. The two canonical evaluation protocols:

### Node classification (transductive).

1. Train embeddings on the full graph (labels not used)
2. Hold out a fraction of node labels
3. Train a simple classifier (logistic regression on  $\mathbf{z}_v$ ) on the visible labels
4. Measure accuracy / F1 on the held-out nodes

### Link prediction.

1. Hide a fraction of edges
2. Train embeddings on the reduced graph
3. For each (held-out edge, random non-edge) pair, score with  $\mathbf{z}_u^\top \mathbf{z}_v$  (or a more elaborate combiner)
4. Measure ROC-AUC

## Visualising embeddings

A 64- or 128-dimensional embedding cannot be plotted directly. Two standard tools:

- **t-SNE** (van der Maaten & Hinton, 2008) — preserves local neighbourhoods; good for visualising clusters
- **UMAP** (McInnes et al., 2018) — preserves local *and* some global structure; faster than t-SNE on large graphs

Both project  $\mathbb{R}^d \rightarrow \mathbb{R}^2$  for visualisation. A typical sanity check after training: project the node embeddings and colour them by community / class label — well-trained embeddings produce visibly separated clusters.

**Caveat.** Distances in t-SNE / UMAP plots are not metrically faithful — use them for qualitative inspection, not quantitative claims.



## Limitations and the road to GNNs

# Limitations of shallow embeddings

## 1. Transductive only

- An embedding is a learned parameter — there is no rule to produce one for a new node
- Adding a node, an edge, or a feature requires retraining (or expensive online updates)
- A blocker for any system whose graph evolves: recommenders, social networks, fraud detection

## 2. Limited structural expressiveness

- Random walks miss certain higher-order patterns (e.g. counting triangles)
- Two non-isomorphic graphs can produce the same walk distribution and therefore the same embeddings

## 3. Node and edge features are ignored

- Real networks come with rich metadata — user profiles, post text, atom types, edge timestamps
- Shallow methods discard all of it; only the binary structure is used
- For feature-rich domains (molecules, knowledge graphs, content recommendation), this is a heavy cost

## Graph Neural Networks — beyond shallow embeddings

The next lecture introduces GNNs, which address each of the limitations above:

- **Richer information processing** — leverage *both* graph structure *and* node/edge features  
In a social network: combine friendship links *with* user profiles and post content  
Critical for molecules, knowledge graphs, and any feature-rich domain
- **Adaptable to dynamic environments** — handle evolving graphs and unseen nodes  
No retraining when the graph changes; the encoder is a function, not a table  
Essential for real-time systems (recommenders, fraud detection, traffic forecasting)
- **Better on complex patterns** — message-passing layers capture higher-order dependencies  
Multiple layers learn hierarchical patterns, much as ConvNets do for images
- **Production-scale** — GNNs at Pinterest, Twitter, Google, Alibaba run on graphs with billions of nodes



# Summary

## Key takeaways

1. **Representation learning automates feature extraction** — embeddings replace the hand-crafted feature engineering pipeline that dominated network ML for two decades
2. **The encoder–decoder framework unifies the field** — every method is a triple of (encoder, similarity, decoder) plus an optimisation objective
3. **Random walks define a flexible, scalable similarity** — they balance local and global structure and are cheap to generate
4. **Skip-gram + negative sampling is the workhorse** — DeepWalk and node2vec both reduce to this NLP-borrowed objective
5. **node2vec's  $p, q$  parameters interpolate** between community-preserving (BFS-like) and role-preserving (DFS-like) embeddings
6. **Shallow embeddings are transductive and feature-blind** — Graph Neural Networks lift both limitations and define the modern frontier

Q&A

